

EnviroGuard ML

Machine Learning PRD — written for Claude to implement and test

3 models · YOLOv8-nano (litter CV) · Prophet noise predictor · Prophet fill predictor

Purpose	ML PRD written as implementation instructions for an AI coding tool (Claude)
Models	3 total — 1 CNN object detection, 2 time-series predictors
Stack	Python · ultralytics (YOLOv8) · prophet · FastAPI · pandas · scikit-learn · matplotlib
Deploy	EC2 instance — FastAPI app serving all 3 models behind 3 endpoints
Test data	TACO dataset (litter CV) · synthetic time-series data (both predictors)
Output	3 working endpoints + evaluation results + prediction graphs

1 — Context and purpose

This document is written as direct implementation instructions for Claude or another AI coding tool. It specifies three ML models for the EnviroGuard environmental sensing system. Read it as: 'here is exactly what to build, in what order, with what data, producing what output.' Every section is actionable.

The three models and what they do

Model	Type	Input	Output	Used for
1 — Litter detector	CNN object detection (YOLOv8-nano)	Camera frame or photo	Severity 1–5 + litter class	Station cam auto-reports + user photo analysis
2 — Noise predictor	Time-series forecast (Prophet)	Historical dB readings + hour/day	24h dB forecast + confidence interval	Health tab timeline + proactive alerts
3 — Fill predictor	Time-series forecast (Prophet)	Historical bin fill % + fill rate	48h fill % forecast + predicted overflow time	Health tab timeline + proactive alerts

What Claude should produce

- A Python project folder with the structure defined in Section 5
- Model 1: YOLOv8-nano fine-tuned on TACO dataset, inference function, accuracy table
- Model 2: Prophet noise model trained on synthetic data, 24h forecast graph PNG
- Model 3: Prophet fill model trained on synthetic data, 48h forecast graph PNG with overflow line
- A FastAPI app (main.py) serving all three models behind three endpoints
- A test suite (test_models.py) that validates all three endpoints with sample inputs
- A README.md with setup instructions, environment variables, and how to run everything

■ All models should work with synthetic/placeholder data first. Real sensor data from the Pi station will replace the synthetic data once hardware is live. Design all data loading functions to accept a CSV path as a parameter — swapping real data in should require changing one variable.

2 — Synthetic data specification

Before real sensor data exists, both Prophet models train on synthetic data. The synthetic data must be realistic — it needs daily and weekly patterns that Prophet can learn from. Claude should generate this data programmatically using the specs below. The data generator should be saved as `generate_data.py` so it can be re-run at any time.

Noise data (`noise_synthetic.csv`)

- Columns: `ds` (datetime, hourly intervals), `y` (float, dB level)
- Duration: 90 days of hourly readings = 2160 rows
- Baseline: 55dB. Add realistic patterns on top:
- Daily pattern: +15dB between 7am–10pm (daytime noise), quieter at night
- Peak events: random spikes to 80–90dB, 3–8 per day, duration 5–15 minutes (simulate buses/trucks)
- Weekly pattern: weekdays 10% louder than weekends
- Random noise: ± 5 dB Gaussian noise on every reading
- Missing values: randomly drop 2% of rows to simulate sensor dropout

Noise data generator code

```
import pandas as pd
import numpy as np
np.random.seed(42)
dates = pd.date_range('2026-01-01', periods=2160, freq='H')
df = pd.DataFrame({'ds': dates})
# Base level + daily pattern
hour = df['ds'].dt.hour
df['y'] = 55 + np.where((hour >= 7) & (hour <= 22), 15, 0)
# Weekly pattern: weekdays louder
df['y'] += np.where(df['ds'].dt.dayofweek < 5, 5, 0)
# Random spikes (buses/trucks)
spike_idx = np.random.choice(len(df), size=500, replace=False)
df.loc[spike_idx, 'y'] += np.random.uniform(20, 35, size=500)
# Gaussian noise
df['y'] += np.random.normal(0, 5, size=len(df))
df['y'] = df['y'].clip(30, 100)
# Drop 2% rows (sensor dropout)
drop_idx = np.random.choice(len(df), size=43, replace=False)
df = df.drop(drop_idx).reset_index(drop=True)
df.to_csv('data/noise_synthetic.csv', index=False)
```

Fill level data (`fill_synthetic.csv`)

- Columns: `ds` (datetime, 15-min intervals), `y` (float, fill percentage 0–100)
- Duration: 90 days of 15-minute readings = 8640 rows
- Pattern: bin starts at 0% every time it overflows (reset), fills gradually
- Fill rate: 2–4% per hour baseline. Spikes to 10–15% per hour at peak times (lunch, evening)
- Peak times: 12pm–2pm and 6pm–9pm on weekdays. 11am–3pm on weekends.
- Overflow event: when `y` reaches 95–100%, reset to 5% (bin has been emptied)
- Overflow frequency: 2–3 times per week at realistic stations
- Random noise: ± 2 % Gaussian on every reading

3 — Model specifications

Model
1

Litter Detector

YOLOv8-nano · CNN Object Detection

Purpose

Detect and classify litter in a camera frame or user-submitted photo. Returns a severity score (1–5), litter class label, confidence score, and bounding box coordinates. Severity is computed from detection count and litter class weights. This output feeds directly into the app's report card UI and the map's trash layer.

Inputs

- Image: JPEG or PNG, any resolution (resized to 640×640 internally)
- Accepted as: base64 string in POST body, or multipart file upload

Outputs

- severity: int 1–5 (1=clean, 5=severe infestation)
- detections: list of {class_name, confidence, bbox: [x,y,w,h]}
- litter_count: total objects detected
- summary: one-sentence plain English description

Dataset

- Primary: TACO dataset — tacodataset.org — 1,500 images, 60 litter classes
- Download via: `pip install fiftyone; import fiftyone.zoo as foz; foz.load_zoo_dataset('taco')`
- Alternative: Roboflow Universe 'Trash Detection' datasets (search roboflow.com/universe)
- Collapse 60 TACO classes into 5 super-classes: plastic, organic, paper, metal, other
- Split: 80% train, 10% val, 10% test — use TACO's recommended 4-fold split

Architecture

YOLOv8-nano (ultralytics). Pre-trained on COCO. Fine-tuned on TACO with 5-class head. YOLOv8-nano chosen for speed (1.9ms inference, 3.2M params) over accuracy — good enough for severity scoring, fast enough for real-time station camera. Input: 640×640. Output: detection boxes + class probs.

Training steps

1. Install: `pip install ultralytics`
2. Download TACO dataset and convert annotations to YOLO format (bounding boxes, not segmentation masks)
3. Create data.yaml: `nc=5, names=[plastic, organic, paper, metal, other], train/val/test paths`
4. Remap TACO's 60 classes to 5 super-classes using a mapping dict before training
5. Fine-tune: `model = YOLO('yolov8n.pt'); model.train(data='data.yaml', epochs=50, imgsz=640, batch=16)`
6. Training takes ~1–2 hours on a GPU (Google Colab free tier works). Save best.pt weights.
7. Run `model.val()` on test split — capture mAP@50 and per-class precision/recall
8. Write inference function: load best.pt, run `predict()`, compute severity from detection count

Evaluation metrics

- mAP@50: target > 0.45 on TACO test split (dataset is challenging — 0.45 is good)

- Per-class precision and recall table — print and save as eval_results.txt
- Severity accuracy: manually label 20 test images 1–5, compare to model output
- Inference speed: time 100 predictions, report mean ms per image

FastAPI endpoint

```
POST /detect-litter Body: { image_b64: string } Returns: { severity, detections, litter_count, summary }
```

Expected file structure

```
models/litter/ best.pt # trained YOLOv8 weights data.yaml # TACO dataset config train.py #
fine-tuning script infer.py # inference function used by FastAPI eval_results.txt # mAP,
precision, recall table severity_map.py # maps detection count + class to severity 1-5
data/taco/ train/images/, train/labels/ val/images/, val/labels/ test/images/, test/labels/
```

Severity scoring logic

The raw model output is bounding boxes and classes. Severity (1–5) needs to be derived. Use this logic in severity_map.py:

```
# Class weights – hazardous litter scores higher CLASS_WEIGHTS = {'plastic': 1.0, 'metal':
1.2, 'paper': 0.8, 'organic': 0.6, 'other': 1.0} def compute_severity(detections: list) ->
int: if not detections: return 1 # clean score = sum(CLASS_WEIGHTS.get(d['class_name'], 1.0)
* d['confidence'] for d in detections) weighted_count = score / max(len(detections), 1) total
= len(detections) if total == 0: return 1 elif total <= 2: return 2 elif total <= 5: return 3
elif total <= 10: return 4 else: return 5
```

Purpose

Predict ambient noise levels (dB) for the next 24 hours at a given station, with hourly granularity and confidence intervals. Output is used by the app's Health tab forecast timeline and by the alert system to fire proactive notifications before a predicted noise threshold is crossed.

Inputs

- Historical noise readings CSV: columns ds (datetime), y (float, dB)
- At inference: station_id (string) — model loads pre-trained .pkl for that station
- Forecast horizon: 24 hours, hourly intervals (configurable)

Outputs

- predictions: list of {hour: ISO timestamp, db: float, lower: float, upper: float}
- peak_hour: timestamp of predicted maximum dB
- peak_db: predicted maximum dB value
- generated_at: ISO timestamp of when forecast was generated

Dataset

- Training: noise_synthetic.csv generated by generate_data.py (see Section 2)
- Format: ds column (hourly datetime), y column (dB float)
- 90 days × 24 hours = ~2,160 rows after dropout simulation
- Real data replacement: Pi station posts dB readings to DynamoDB every 5 sec; aggregate to hourly mean before training
- Retrain: weekly cron job on EC2 retrains on latest 90 days of real data

Architecture

Facebook Prophet (prophet Python package). Additive model: trend + daily seasonality + weekly seasonality + holiday effects. Chosen for: interpretable components, handles missing values natively, built-in confidence intervals, fast training (seconds not hours), no GPU needed. Well-validated for environmental time-series (see literature: SARIMA vs Prophet studies show Prophet competitive on urban noise/AQ).

Training steps

1. Install: pip install prophet pandas matplotlib scikit-learn
2. Load synthetic CSV into DataFrame with columns ds and y
3. Instantiate: model = Prophet(daily_seasonality=True, weekly_seasonality=True, interval_width=0.80)
4. Fit: model.fit(df) — trains in seconds on 90 days of hourly data
5. Generate future DataFrame: future = model.make_future_dataframe(periods=24, freq='H')
6. Forecast: forecast = model.predict(future) — returns yhat, yhat_lower, yhat_upper
7. Save model: import pickle; pickle.dump(model, open('models/noise/noise_model.pkl', 'wb'))
8. Generate and save forecast plot: fig = model.plot(forecast); fig.savefig('outputs/noise_forecast.png')
9. Generate and save components plot: fig2 = plot_components(model, forecast); fig2.savefig('outputs/noise_components.png')

Evaluation metrics

- Hold-out eval: train on first 80 days, test on last 10 days
- MAE: `mean_absolute_error(actual_db, predicted_db)` — target < 8 dB
- RMSE: `sqrt(mean_squared_error(actual, predicted))` — target < 12 dB
- Coverage: % of actual values falling within `yhat_lower/yhat_upper` — target > 75%
- Use Prophet's built-in: `cross_validation(model, initial='60 days', period='7 days', horizon='24 hours')`
- Print `performance_metrics(df_cv)[['mae','rmse','coverage']]` and save to `eval_noise.txt`

FastAPI endpoint

```
GET /predict-noise/{station_id} Returns: { predictions: [{hour, db, lower, upper}], peak_hour,
peak_db, generated_at }
```

Expected file structure

```
models/noise/ noise_model.pkl # serialized Prophet model train_noise.py # training script
infer_noise.py # inference function used by FastAPI eval_noise.txt # MAE, RMSE, coverage
results outputs/ noise_forecast.png # 24h forecast plot noise_components.png # trend +
seasonality components data/ noise_synthetic.csv # generated by generate_data.py
generate_data.py # synthetic data generator
```

Purpose

Predict bin fill percentage for the next 48 hours at a given station, with 15-minute granularity. Critically, derive the predicted overflow time — when the bin is expected to reach 95% full. This is the most actionable output in the system: it enables proactive sanitation alerts before overflow happens, not after.

Inputs

- Historical fill readings CSV: columns ds (datetime, 15-min intervals), y (float, fill %)
- At inference: station_id (string) — loads pre-trained .pkl for that station
- Overflow threshold: default 95% (configurable per station)

Outputs

- predictions: list of {hour: ISO timestamp, fill_pct: float, lower: float, upper: float}
- overflow_at: ISO timestamp of first predicted breach of 95% threshold (null if not predicted)
- hours_until_overflow: int (null if no overflow predicted in window)
- current_fill_pct: latest known reading passed through from sensor data
- generated_at: ISO timestamp

Dataset

- Training: fill_synthetic.csv generated by generate_data.py (see Section 2)
- Format: ds column (15-min datetime), y column (fill % float, 0–100)
- 90 days × 96 readings/day = ~8,640 rows after noise simulation
- Real data: Pi ultrasonic sensor posts fill % to DynamoDB every 60 sec; use as-is (no aggregation needed)
- Retrain: weekly cron job retrains on latest 90 days of real readings per station

Architecture

Facebook Prophet with 15-minute frequency. Same architecture as noise model but with freq='15T' instead of hourly. Key difference: overflow detection logic runs post-forecast — scan yhat for first crossing of threshold. Prophet handles the sawtooth pattern (fill-empty-fill) well because it models trend changepoints automatically. Set changepoint_prior_scale=0.5 for more flexibility to catch rapid fill events.

Training steps

1. Load fill_synthetic.csv into DataFrame with ds and y columns
2. Instantiate: model = Prophet(daily_seasonality=True, weekly_seasonality=True, changepoint_prior_scale=0.5, interval_width=0.80)
3. Fit: model.fit(df)
4. Make future: future = model.make_future_dataframe(periods=192, freq='15T') # 48 hours
5. Forecast: forecast = model.predict(future)
6. Save model: pickle.dump(model, open('models/fill/fill_model.pkl', 'wb'))
7. Overflow detection: scan forecast['yhat'] for first value >= 95; record that timestamp as overflow_at
8. Plot forecast: add horizontal red line at y=95 (overflow threshold); save as outputs/fill_forecast.png
9. Label x-axis clearly with overflow_at timestamp marked with a vertical dashed line

Evaluation metrics

- Hold-out: train on 80 days, test on last 10 days at 15-min resolution
- MAE on fill %: target < 5 percentage points
- Overflow time accuracy: for overflow events in test set, compare predicted vs actual overflow time — target within 2 hours
- Use Prophet cross_validation with horizon='48 hours', period='7 days', initial='60 days'
- Print performance_metrics(df_cv)[['mae','rmse','coverage']] — save to eval_fill.txt

FastAPI endpoint

```
GET /predict-fill/{station_id} Returns: { predictions: [{hour, fill_pct, lower, upper}],  
overflow_at, hours_until_overflow, current_fill_pct, generated_at }
```

Expected file structure

```
models/fill/ fill_model.pkl # serialized Prophet model train_fill.py # training script  
infer_fill.py # inference function used by FastAPI eval_fill.txt # MAE, RMSE, overflow  
accuracy outputs/ fill_forecast.png # 48h fill % plot with overflow line fill_components.png  
# trend + seasonality components data/ fill_synthetic.csv # generated by generate_data.py
```

4 — FastAPI application

All three models are served by a single FastAPI app (main.py). The app loads all three models on startup and keeps them in memory. Each endpoint is independently callable. The app runs with uvicorn on port 8000.

main.py structure

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import pickle, base64
from ultralytics import YOLO

app = FastAPI(title='EnviroGuard ML API')

# Load models on startup – all stay in memory
yolo_model = YOLO('models/litter/best.pt')
noise_model = pickle.load(open('models/noise/noise_model.pkl', 'rb'))
fill_model = pickle.load(open('models/fill/fill_model.pkl', 'rb'))

class ImageInput(BaseModel):
    image_b64: str

@app.post('/detect-litter')
def detect_litter(body: ImageInput):
    # decode base64 → temp file → YOLO predict → compute severity ...
    @app.get('/predict-noise/{station_id}')
    def predict_noise(station_id: str):
        # load station model or use default → make_future_dataframe → predict → return JSON ...
    @app.get('/predict-fill/{station_id}')
    def predict_fill(station_id: str):
        # load station model → predict 48h → scan for overflow → return JSON ...
    @app.get('/health')
    def health():
        return {'status': 'ok', 'models_loaded': 3}
```

Running the server

- Install deps: `pip install fastapi uvicorn ultralytics prophet pandas scikit-learn matplotlib pillow`
- Run locally: `uvicorn main:app --host 0.0.0.0 --port 8000 --reload`
- On EC2: `uvicorn main:app --host 0.0.0.0 --port 8000` (run with `nohup` or `systemd` service)
- Health check: GET `http://localhost:8000/health` should return `{'status': 'ok', 'models_loaded': 3}`
- Interactive docs: `http://localhost:8000/docs` (FastAPI auto-generates Swagger UI)

5 — Test suite specification

Claude must write a test file (test_models.py) using pytest. Tests run against a locally running FastAPI server. All tests must pass before the models are considered done.

Test name	What it checks	Pass condition
test_litter_clean_image	POST /detect-litter with a photo of an empty pavement	severity == 1, detections == []
test_litter_cluttered_image	POST /detect-litter with a photo of clear litter	severity >= 3, len(detections) > 0
test_litter_base64_format	POST /detect-litter with valid base64 PNG	Response 200, all required fields present
test_litter_invalid_input	POST /detect-litter with empty string	Response 422 (validation error)
test_noise_returns_24_hours	GET /predict-noise/s01	len(predictions) == 24
test_noise_confidence_interval	GET /predict-noise/s01	All lower <= db <= upper
test_noise_peak_hour_present	GET /predict-noise/s01	peak_hour and peak_db present, peak_db > 0
test_fill_returns_48_hours	GET /predict-fill/s01	len(predictions) == 192 (48h × 4 per hour)
test_fill_overflow_detected	GET /predict-fill/s01 (model trained on data with overflow)	overflow_at is not null, hours_until_overflow > 0
test_fill_values_bounded	GET /predict-fill/s01	All fill_pct between 0 and 100
test_health_endpoint	GET /health	{'status': 'ok', 'models_loaded': 3}
test_all_endpoints_fast	All 3 endpoints timed	Each responds in < 5 seconds

Running the tests

```
# In terminal 1 – start the server uvicorn main:app --host 0.0.0.0 --port 8000 # In terminal 2
– run tests pip install pytest httpx pytest test_models.py -v # All tests should show PASSED #
Save output to test_results.txt: pytest test_models.py -v > test_results.txt
```

6 — Full project file structure

Claude should produce exactly this folder structure. Every file listed below must exist and be functional.

```
enviroguard-ml/ ■■■ main.py # FastAPI app – serves all 3 endpoints ■■■ test_models.py #
pytest test suite ■■■ requirements.txt # all Python dependencies with versions ■■■ README.md
# setup, running, endpoint reference ■■■ .env.example # EC2_HOST, PORT – no secrets ■ ■■■
data/ ■ ■■■ generate_data.py # generates noise_synthetic.csv + fill_synthetic.csv ■ ■■■
noise_synthetic.csv # 90 days hourly dB readings ■ ■■■ fill_synthetic.csv # 90 days 15-min
fill % readings ■ ■■■ taco/ # TACO dataset (downloaded separately) ■ ■■■ train/, val/,
test/ (images + labels in YOLO format) ■ ■■■ data.yaml ■ ■■■ models/ ■ ■■■ litter/ ■ ■
■■■ best.pt # YOLOv8 fine-tuned weights ■ ■ ■■■ train.py # training script ■ ■ ■■■
infer.py # inference + severity logic ■ ■ ■■■ severity_map.py # detection count → severity
1-5 ■ ■ ■■■ eval_results.txt # mAP, precision, recall ■ ■■■ noise/ ■ ■ ■■■ noise_model.pkl
# serialized Prophet model ■ ■ ■■■ train_noise.py # training + evaluation script ■ ■ ■■■
infer_noise.py # 24h forecast function ■ ■ ■■■ eval_noise.txt # MAE, RMSE, coverage ■ ■■■
fill/ ■ ■■■ fill_model.pkl # serialized Prophet model ■ ■■■ train_fill.py # training +
evaluation script ■ ■■■ infer_fill.py # 48h forecast + overflow detection ■ ■■■
eval_fill.txt # MAE, overflow accuracy ■ ■■■ outputs/ ■■■ noise_forecast.png # 24h noise
prediction graph ■■■ noise_components.png # Prophet trend + seasonality ■■■
fill_forecast.png # 48h fill prediction + overflow line ■■■ fill_components.png # Prophet
trend + seasonality ■■■ test_results.txt # pytest output
```

7 — README specification

The README.md must contain these sections in this order:

EnviroGuard ML

One paragraph: what the three models do and how they fit into the EnviroGuard system.

Quick start

4 commands: clone, pip install -r requirements.txt, python data/generate_data.py, uvicorn main:app

Training the models

One subsection per model. Command to run each training script. Expected output.

Running the tests

pytest command, expected output, what PASSED means.

API reference

Table: Method, Path, Description, Request body, Response shape. All 3 endpoints + /health.

Replacing synthetic data with real sensor data

Exact instructions: CSV format required, which file path to update, how to retrain.

Deploying to EC2

Step by step: SSH in, clone repo, install deps, run uvicorn with nohup, verify with /health.

For AI tools

Section explicitly addressed to Claude or other AI tools picking this up: stack summary, conventions used, known limitations, what still needs to be done.

EnviroGuard ML

ML PRD — written for Claude to implement

Python · ultralytics (YOLOv8-nano) · prophet · FastAPI · pandas · scikit-learn · matplotlib · pytest